Team Reference ICPC

Nombre del equipo — Universidad

Convenciones. Todo el código es C++17 y asume `#include <bits/stdc++.h>` y `using namespace std;` (ver la plantilla). No se usan macros: cada bloque se copia tal cual. Los nombres del código están en inglés y los comentarios en español. Los índices empiezan en 0. Cuando un bloque dice *Requiere*, hay que copiar también ese otro bloque. Cada algoritmo fue probado contra una solución de fuerza bruta con casos aleatorios (carpeta `tests`).

Índice

1 Plantilla y utilidades **1**
- 1.1 Plantilla . . . 1
- 1.2 Números aleatorios . . . 1
- 1.3 Stress test (bash) . . . 2
- 1.4 Búsqueda binaria (lower_bound y upper_bound) . . . 2

2 Estructuras de datos **2**
- 2.1 Segment Tree 4N con lazy . . . 2
- 2.2 Segment Tree 2N iterativo . . . 3
- 2.3 Segment Tree recursivo (update puntual y lazy) . . . 3
- 2.4 Merge Sort Tree (k-ésimo menor en rango) . . . 4
- 2.5 Fenwick Tree . . . 5
- 2.6 Sparse Table . . . 5
- 2.7 Union-Find (DSU) . . . 5
- 2.8 Sqrt Decomposition . . . 6
- 2.9 Algoritmo de Mo . . . 6
- 2.10 Heavy-Light Decomposition (y LCA) . . . 6
- 2.11 Centroid Decomposition . . . 7
- 2.12 Ordered Set (pb_ds) . . . 8
- 2.13 Li Chao Tree . . . 8

3 Grafos **8**
- 3.1 Dijkstra . . . 8
- 3.2 Biconexas, puentes, articulación, bridge tree . . . 8
- 3.3 Camino y circuito euleriano (Hierholzer) . . . 9
- 3.4 SCC y grafo reducido . . . 10
- 3.5 2-SAT . . . 10
- 3.6 Flujo máximo (Dinic) . . . 11
- 3.7 Flujo máximo de costo mínimo . . . 11
- 3.8 Húngaro (asignación de peso mín/máx) . . . 12
- 3.9 Matching bipartito (Hopcroft-Karp) . . . 12

4 Matemática **13**
- 4.1 Aritmética modular . . . 13
- 4.2 Miller-Rabin . . . 13
- 4.3 Pollard Rho . . . 13
- 4.4 Euclides extendido, inverso, diofántica . . . 13
- 4.5 Teorema chino del resto . . . 14
- 4.6 Baby Step / Giant Step . . . 14
- 4.7 Criba lineal . . . 14
- 4.8 Gauss (reales y módulo primo) . . . 14
- 4.9 FFT . . . 15
- 4.10 NTT . . . 16
- 4.11 Matrices . . . 16
- 4.12 Combinatoria . . . 16

5 Strings **17**
- 5.1 Hashing . . . 17
- 5.2 KMP . . . 17
- 5.3 Función Z . . . 17
- 5.4 Manacher . . . 17
- 5.5 Suffix Automaton (DP en el DAG) . . . 18
- 5.6 Suffix Array y LCP . . . 19
- 5.7 Aho-Corasick . . . 19

6 Geometría **20**
- 6.1 Punto y primitivas . . . 20
- 6.2 Intersección de segmentos . . . 20
- 6.3 Convex Hull . . . 20
- 6.4 Puntos antipodales (rotating calipers) . . . 21
- 6.5 Círculo mínimo . . . 21
- 6.6 Punto en polígono . . . 21
- 6.7 Par de puntos más cercano . . . 22

7 Fórmulas y recordatorios **22**

1. Plantilla y utilidades

1.1 Plantilla

```cpp
// Compilar: g++ -std=c++17 -O2 -Wall -Wextra -Wshadow a.cpp -o a
// Depurar: agregar -g -fsanitize=address,undefined
//          -D_GLIBCXX_DEBUG
#include <bits/stdc++.h>
using namespace std;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  return 0;
}
```

1.2 Números aleatorios

```cpp
// Numeros aleatorios de buena calidad (no usar rand()).
mt19937_64 rng(
    chrono::steady_clock::now().time_since_epoch().count());
long long random_int(long long l, long long r) {  // en [l, r]
  return uniform_int_distribution<long long>(l, r)(rng);
}
```

```cpp
// shuffle(v.begin(), v.end(), rng);
```

1.3 Stress test (bash)

```bash
# Stress test: sol = solucion, brute = fuerza bruta,
# gen = generador que recibe la semilla como argumento.
for ((i = 1; ; i++)); do
  ./gen $i > in.txt
  ./sol < in.txt > out1.txt
  ./brute < in.txt > out2.txt
  if ! cmp -s out1.txt out2.txt; then
    echo "Failed on seed $i (see in.txt)"
    break
  fi
  echo "ok $i"
done
```

1.4 Búsqueda binaria (lower_bound y upper_bound)

```cpp
// Busqueda binaria. El arreglo debe estar ORDENADO de menor a
// mayor. Indices en [0, n). Cada busqueda es O(log n).
//
// lower_bound(x): primer indice i con a[i] >= x (n si no hay).
//   Es donde se insertaria x ANTES de todos sus iguales.
// upper_bound(x): primer indice i con a[i] > x (n si no hay).
//   Es donde se insertaria x DESPUES de todos sus iguales.
//
// Ejemplo con a = {1, 3, 3, 3, 7} (n = 5):
//   lower_bound(3) = 1    upper_bound(3) = 4   (los 3 son [1, 4))
//   lower_bound(4) = 4    upper_bound(4) = 4   (4 no esta)
//   lower_bound(0) = 0    upper_bound(9) = 5 = n
//
// Lo que se obtiene de ellas (lb = lower_bound, ub = upper_bound):
//   cantidad de veces que aparece x  = ub(x) - lb(x)
//   x aparece                        <=> lb(x) < n && a[lb(x)] == x
//   cantidad de elementos < x         = lb(x)
//   cantidad de elementos <= x        = ub(x)
//   cantidad de elementos en [l, r]   = ub(r) - lb(l)
//   ultimo indice con a[i] < x        = lb(x) - 1   (-1 si no hay)
//   ultimo indice con a[i] <= x       = ub(x) - 1   (-1 si no hay)

// Invariante: todo indice < lo es "chico" (a[i] < x) y todo
// indice >= hi es "grande" (a[i] >= x). Al terminar lo == hi.
int lower_bound_manual(const vector<long long>& a, long long x) {
  int lo = 0, hi = a.size();
  while (lo < hi) {
    int mid = lo + (hi - lo) / 2;
    if (a[mid] >= x) hi = mid;  // mid cumple: la respuesta es <= mid
    else lo = mid + 1;          // mid no cumple: descartarlo
  }
  return lo;
}
// Igual, pero la condicion es estricta: a[mid] > x.
int upper_bound_manual(const vector<long long>& a, long long x) {
  int lo = 0, hi = a.size();
  while (lo < hi) {
    int mid = lo + (hi - lo) / 2;
    if (a[mid] > x) hi = mid;
    else lo = mid + 1;
  }
  return lo;
}

// Con la STL (devuelven iteradores; restar begin() da el indice):
//   int lb = lower_bound(a.begin(), a.end(), x) - a.begin();
//   int ub = upper_bound(a.begin(), a.end(), x) - a.begin();
//   En set/map usar s.lower_bound(x): O(log n). En cambio
//   lower_bound(s.begin(), s.end(), x) sobre un set es O(n).
//   Arreglo en orden decreciente: pasar greater<long long>() como
//   cuarto argumento (lb = primer a[i] <= x, ub = primer a[i] < x).

// Busqueda binaria sobre la respuesta. ok(m) debe ser monotono en
// [lo, hi]: falso, ..., falso, verdadero, ..., verdadero.
// Retorna el MENOR m con ok(m), o hi + 1 si ninguno cumple.
// lower_bound es el caso ok(i) = a[i] >= x; upper_bound, a[i] > x.
// Para el MAYOR m con ok(m) cuando el patron es verdadero...falso:
//   first_true(lo, hi, [&](long long m) { return !ok(m); }) - 1
template <class F>
long long first_true(long long lo, long long hi, F ok) {
  hi++;  // hi pasa a significar "ninguno cumple"
  while (lo < hi) {
    long long mid = lo + (hi - lo) / 2;
    if (ok(mid)) hi = mid;
    else lo = mid + 1;
  }
  return lo;
}

// Version en reales: menor x en [lo, hi] con ok(x) (ok monotono).
// Iterar un numero fijo de veces evita problemas con EPS:
// 100 iteraciones achican el intervalo en un factor de 2^100.
template <class F>
double bisection(double lo, double hi, F ok) {
  for (int it = 0; it < 100; it++) {
    double mid = (lo + hi) / 2;
    if (ok(mid)) hi = mid;
    else lo = mid;
  }
  return hi;
}
```

2. Estructuras de datos

2.1 Segment Tree 4N con lazy

```cpp
// Segment Tree recursivo (arreglo de 4N) con lazy propagation.
// update(l, r, x): suma x a todo [l, r].
// query(l, r): suma de [l, r]. Indices 0..n-1, rangos cerrados.
// O(log n) por operacion.
// Para min/max: cambiar la combinacion y el neutro de query, y
// en apply() usar t[v] += x (sin multiplicar por el largo).
struct SegTree {
  int n;
```

```
  vector<long long> t, lz;
  SegTree(int n_) : n(n_), t(4 * n_, 0), lz(4 * n_, 0) {}
  SegTree(const vector<long long>& a) : SegTree((int)a.size()) {
    build(1, 0, n - 1, a);
  }
  void build(int v, int tl, int tr, const vector<long long>& a) {
    if (tl == tr) { t[v] = a[tl]; return; }
    int mid = (tl + tr) / 2;
    build(2 * v, tl, mid, a);
    build(2 * v + 1, mid + 1, tr, a);
    t[v] = t[2 * v] + t[2 * v + 1];
  }
  void apply(int v, int tl, int tr, long long x) {
    t[v] += x * (tr - tl + 1);
    lz[v] += x;
  }
  void push(int v, int tl, int tr) {
    if (lz[v] == 0) return;
    int mid = (tl + tr) / 2;
    apply(2 * v, tl, mid, lz[v]);
    apply(2 * v + 1, mid + 1, tr, lz[v]);
    lz[v] = 0;
  }
  void update(int v, int tl, int tr, int l, int r, long long x) {
    if (r < tl || tr < l) return;
    if (l <= tl && tr <= r) { apply(v, tl, tr, x); return; }
    push(v, tl, tr);
    int mid = (tl + tr) / 2;
    update(2 * v, tl, mid, l, r, x);
    update(2 * v + 1, mid + 1, tr, l, r, x);
    t[v] = t[2 * v] + t[2 * v + 1];
  }
  long long query(int v, int tl, int tr, int l, int r) {
    if (r < tl || tr < l) return 0;
    if (l <= tl && tr <= r) return t[v];
    push(v, tl, tr);
    int mid = (tl + tr) / 2;
    return query(2 * v, tl, mid, l, r) +
           query(2 * v + 1, mid + 1, tr, l, r);
  }
  void update(int l, int r, long long x) {
    update(1, 0, n - 1, l, r, x);
  }
  long long query(int l, int r) {
    return query(1, 0, n - 1, l, r);
  }
};
```

2.2 Segment Tree 2N iterativo

```
// Segment Tree iterativo (arreglo de 2N), sin recursion.
// update(p, v): asigna a[p] = v.  query(l, r): rango [l, r).
// O(log n). Cambiar IDENTITY y combine() para min, max, gcd...
struct SegTree2N {
  int n;
  vector<long long> t;
  long long IDENTITY = 0;
  long long combine(long long a, long long b) { return a + b; }
  SegTree2N(const vector<long long>& a)
      : n(a.size()), t(2 * a.size()) {
    for (int i = 0; i < n; i++) t[n + i] = a[i];
    for (int i = n - 1; i >= 1; i--)
      t[i] = combine(t[2 * i], t[2 * i + 1]);
  }
  void update(int p, long long v) {
    p += n;
    t[p] = v;
    for (p /= 2; p >= 1; p /= 2)
      t[p] = combine(t[2 * p], t[2 * p + 1]);
  }
  long long query(int l, int r) {  // [l, r)
    long long res_l = IDENTITY, res_r = IDENTITY;
    for (l += n, r += n; l < r; l /= 2, r /= 2) {
      if (l & 1) res_l = combine(res_l, t[l++]);
      if (r & 1) res_r = combine(t[--r], res_r);
    }
    return combine(res_l, res_r);
  }
};
```

2.3 Segment Tree recursivo (update puntual y lazy)

```
// Segment Tree recursivo con arreglos globales (version del equipo).
// Parte 1: update puntual + query de suma en rango.
// Parte 2 (namespace lazy): update de suma en rango + query.
// Indices 0..n-1, rangos cerrados [l, r]. O(log n) por operacion.
// IMPORTANTE: dimensionar los vectores DESPUES de leer n (si se
// hace al declararlos, n todavia vale 0 y quedan vacios):
//   s.resize(n); /* leer s */ st.assign(4 * n, 0);
//   build(1, 0, n - 1);
//   query(1, 0, n - 1, l, r);    update(1, 0, n - 1, val, pos);
//   lazy::n = n; lazy::s = s;
//   lazy::st.assign(4 * n, 0); lazy::lazy.assign(4 * n, 0);
//   lazy::build(1, 0, n - 1);
//   lazy::update(1, 0, n - 1, l, r, val);
//   lazy::query(1, 0, n - 1, l, r);
// Las sumas son int: si pueden pasar de ~2e9, usar long long.

int n; //tamano del arreglo original
vector<int>s;//arreglo original
vector<int>st;//segment tree (4*n nodos)

void build(int v,int l,int r){
    if(l==r){
        st[v]=s[l];
    }
    else{
        int m=(l+r)/2;
        build(2*v,l,m);
        build(2*v+1,m+1,r);
        // en este caso es una suma pero puede ser cualquier
        // operacion asociativa (a o b) o c = a o (b o c)
```

```
        st[v]=st[2*v]+st[2*v+1];
    }
}

// v=nodo, tl=posicion izquierda en el st, tr=posicion derecha
// en el st, l y r rango de la query
int query(int v,int tl,int tr,int l,int r){
 if(tl>r || tr<l){
    // cero porque es el neutro de la suma; si fuera otra
    // operacion hay que poner su neutro
    return 0;
 }
 if(tl>=l && tr<=r){
    return st[v];
 }
 int tm=(tl+tr)/2;
 return query(2*v,tl,tm,l,r)+query(2*v+1,tm+1,tr,l,r);

}

void update(int v,int tl,int tr,int val,int pos){
    if(tl==tr){
        st[v]=val;
        return;
    }
    int tm=(tl+tr)/2;
    if(pos<=tm)update(2*v,tl,tm,val,pos);
    else update(2*v+1,tm+1,tr,val,pos);
    st[v]=st[2*v]+st[2*v+1];
}


//********************** LAZY PROPAGATION **********************/
namespace lazy{
int n;
vector<int>s;
vector<int> st;
vector<int>lazy;

void build(int v,int l,int r){
    if(l==r){
        st[v]=s[l];
    }
    else{
        int m=(l+r)/2;
        build(2*v,l,m);
        build(2*v+1,m+1,r);
        // en este caso es una suma pero puede ser cualquier
        // operacion asociativa (a o b) o c = a o (b o c)
        st[v]=st[2*v]+st[2*v+1];
    }
}

// aplica el valor al nodo y apunta lo que se aumento para sus hijos
void apply(int v, int l, int r, int val){
    st[v]+=(r-l+1)*val;
    lazy[v]+=val;
}

void push(int v, int l, int r){//baja el valor pendiente a sus hijos
    if (lazy[v]==0){
        return;
    }
    int tm =(l+r)/2;
    apply(2*v,l,tm,lazy[v]);
    apply(2*v+1,tm+1,r,lazy[v]);
    lazy[v]=0;

}
void update(int v,int tl,int tr,int l,int r,int val){
    if(tl>r||tr<l){
        return;
    }
    if(tl>=l&&tr<=r){
        apply(v,tl,tr,val);
        return;
    }
    push(v,tl,tr);
    int tm =(tl+tr)/2;
    update(2*v,tl,tm,l,r,val);
    update(2*v+1,tm+1,tr,l,r,val);
    st[v]=st[2*v]+st[2*v+1];
}
int query(int v, int tl , int tr,int l, int r){
    if(tl>r||tr<l){
        return 0;
    }
    if(tl>=l && tr<=r){
        return st[v];
    }
    push(v,tl,tr);
    int tm= (tl+tr)/2;
    return query(2*v,tl,tm,l,r)+query(2*v+1,tm+1,tr,l,r);
}
}
```

2.4 Merge Sort Tree (k-ésimo menor en rango)

```
// Merge Sort Tree (version del equipo). Cada nodo guarda ORDENADOS
// los valores de su rango. Memoria y build O(n log n).
// lesskquery(1, 0, n - 1, l, r, value): cuantos s[i] <= value con
//   i en [l, r]. O(log^2 n).
// k_th_smoller(l, r, k): k-esimo menor de s[l..r], con k desde 1 y
//   1 <= k <= r - l + 1. O(log^3 n).
// Uso: mergesort::n = n; mergesort::s = arreglo;
//      mergesort::build(1, 0, n - 1);
// Requiere n <= MAXN. Para otro caso de prueba basta volver a
// llamar build.
namespace mergesort{
```

```
    int n;
    const int  MAXN = 100005;
    vector<int>s;
    vector<int>st[4*MAXN];

    void build(int node,int l,int r){
        if (l==r){
            st[node]={s[l]};
        }
        else{
            int m=(l+r)/2;
            build(2*node,l,m);
            build(2*node+1,m+1,r);

            st[node].resize(r-l+1);
            merge(st[2*node].begin(),st[2*node].end(),
                  st[2*node+1].begin(),st[2*node+1].end(),
                  st[node].begin());

        }
    }

    int lesskquery(int v, int tl,int tr,int l,int r,int value){
        if(tl>r||tr<l){
            return 0;
        }
        if(tl>=l && tr<=r){
            // aqui busco los menores iguales; si quisiera los
            // menores estrictos uso lower_bound
            return upper_bound(st[v].begin(),st[v].end(),value)-st[v].begin();

        }
        int tm=(tl+tr)/2;
        return lesskquery(2*v,tl,tm,l,r,value)+lesskquery(2*v+1,tm+1,tr,l,r,value);
    }


    int k_th_smoller(int ql,int qr,int k){
        int lo=0;
        int hi=n-1;
        while(lo<hi){
            int mid=(lo+hi)/2;
            if(lesskquery(1,0,n-1,ql,qr,st[1][mid])>=k){
                hi=mid;
            }
            else{

            lo=mid+1;
        }
        }
        return st[1][lo];
    }

}
```

2.5 Fenwick Tree

```
// Fenwick Tree (BIT). add y sum en O(log n). Indices 0..n-1.
struct Fenwick {
  int n;
  vector<long long> bit;
  Fenwick(int n_) : n(n_), bit(n_ + 1, 0) {}
  void add(int i, long long v) {
    for (i++; i <= n; i += i & -i) bit[i] += v;
  }
  long long sum(int i) {  // suma de [0, i]
    long long s = 0;
    for (i++; i > 0; i -= i & -i) s += bit[i];
    return s;
  }
  long long sum(int l, int r) { return sum(r) - sum(l - 1); }
  // Menor i con sum(i) >= k (valores no negativos). n si no hay.
  int lower_bound(long long k) {
    int pos = 0, pw = 1;
    while (2 * pw <= n) pw *= 2;
    for (; pw > 0; pw /= 2)
      if (pos + pw <= n && bit[pos + pw] < k) {
        pos += pw;
        k -= bit[pos];
      }
    return pos;
  }
};
```

2.6 Sparse Table

```
// Sparse Table: minimo en rango (estatico).
// Construccion O(n log n), consulta O(1). Rango cerrado [l, r].
struct SparseTable {
  vector<vector<int>> t;
  SparseTable(const vector<int>& a) {
    int n = a.size(), levels = 1;
    while ((1 << levels) <= n) levels++;
    t.assign(levels, a);
    for (int k = 1; k < levels; k++)
      for (int i = 0; i + (1 << k) <= n; i++)
        t[k][i] = min(t[k - 1][i], t[k - 1][i + (1 << (k - 1))]);
  }
  int query(int l, int r) {
    int k = 31 - __builtin_clz(r - l + 1);
    return min(t[k][l], t[k][r - (1 << k) + 1]);
  }
};
```

2.7 Union-Find (DSU)

```
// Union-Find con compresion de caminos y union por tamano.
// Casi O(1) amortizado por operacion.
struct DSU {
  vector<int> parent, sz;
  int comps;
  DSU(int n) : parent(n), sz(n, 1), comps(n) {
```

```
    iota(parent.begin(), parent.end(), 0);
  }
  int find(int x) {
    return parent[x] == x ? x : parent[x] = find(parent[x]);
  }
  bool unite(int a, int b) {
    a = find(a);
    b = find(b);
    if (a == b) return false;
    if (sz[a] < sz[b]) swap(a, b);
    parent[b] = a;
    sz[a] += sz[b];
    comps--;
    return true;
  }
};
```

2.8 Sqrt Decomposition

```
// Sqrt Decomposition: suma en rango con suma perezosa por bloque.
// update(l, r, x) y query(l, r) en O(sqrt n). Rangos cerrados.
struct SqrtDecomp {
  int n, B;
  vector<long long> a, block_sum, lazy;
  SqrtDecomp(const vector<long long>& v) : n(v.size()), a(v) {
    B = max(1, (int)sqrt(n));
    block_sum.assign(n / B + 1, 0);
    lazy.assign(n / B + 1, 0);
    for (int i = 0; i < n; i++) block_sum[i / B] += a[i];
  }
  void add_naive(int l, int r, long long x) {  // dentro de un bloque
    for (int i = l; i <= r; i++) {
      a[i] += x;
      block_sum[i / B] += x;
    }
  }
  long long sum_naive(int l, int r) {  // dentro de un bloque
    long long res = 0;
    for (int i = l; i <= r; i++) res += a[i] + lazy[i / B];
    return res;
  }
  void update(int l, int r, long long x) {
    int bl = l / B, br = r / B;
    if (bl == br) { add_naive(l, r, x); return; }
    add_naive(l, (bl + 1) * B - 1, x);
    for (int b = bl + 1; b < br; b++) lazy[b] += x;
    add_naive(br * B, r, x);
  }
  long long query(int l, int r) {
    int bl = l / B, br = r / B;
    if (bl == br) return sum_naive(l, r);
    long long res = sum_naive(l, (bl + 1) * B - 1) + sum_naive(br * B, r);
    for (int b = bl + 1; b < br; b++) res += block_sum[b] + lazy[b] * B;
    return res;
  }
};
```

2.9 Algoritmo de Mo

```
// Algoritmo de Mo: consultas offline en O((N + Q) sqrt N).
// Ejemplo: cantidad de valores distintos en [l, r] (cerrado).
// Para otro problema solo cambian add_pos() y remove_pos().
// Los valores de a deben ser >= 0 y chicos; si no, comprimirlos.
struct Query { int l, r, id; };

vector<int> mo(const vector<int>& a, vector<Query> qs) {
  if (qs.empty()) return {};
  int n = a.size();
  int B = max(1, (int)(n / sqrt((double)qs.size())));
  sort(qs.begin(), qs.end(),
       [&](const Query& x, const Query& y) {
    int bx = x.l / B, by = y.l / B;
    if (bx != by) return bx < by;
    return bx % 2 == 0 ? x.r < y.r : x.r > y.r;
  });
  vector<int> cnt(*max_element(a.begin(), a.end()) + 1, 0);
  vector<int> res(qs.size());
  int distinct = 0, L = 0, R = -1;
  auto add_pos = [&](int i) { if (cnt[a[i]]++ == 0) distinct++; };
  auto remove_pos = [&](int i) { if (--cnt[a[i]] == 0) distinct--; };
  for (const Query& q : qs) {
    while (L > q.l) add_pos(--L);
    while (R < q.r) add_pos(++R);
    while (L < q.l) remove_pos(L++);
    while (R > q.r) remove_pos(R--);
    res[q.id] = distinct;
  }
  return res;
}
```

2.10 Heavy-Light Decomposition (y LCA)

```
// Heavy-Light Decomposition. Requiere SegTree (lazy).
// Valores en los vertices. Uso: add_edge de las n-1 aristas y
// luego build(root). Con hld.lca(u, v) ya se tiene el LCA.
// Camino: O(log^2 n). Subarbol y LCA: O(log n).
// Valores en aristas: guardar cada arista en su vertice hijo y
// en el ultimo tramo de path() usar f(pos[u] + 1, pos[v]).
struct HLD {
  int n, timer = 0;
  vector<vector<int>> adj;
  vector<int> parent, depth, sz, heavy, head, pos;
  SegTree st;
  HLD(int n_) : n(n_), adj(n_), parent(n_), depth(n_), sz(n_),
                heavy(n_), head(n_), pos(n_), st(n_) {}
  void add_edge(int u, int v) {
    adj[u].push_back(v);
    adj[v].push_back(u);
  }
  void dfs_size(int u) {
    sz[u] = 1;
    heavy[u] = -1;
    for (int v : adj[u]) {
```

```
    if (v == parent[u]) continue;
    parent[v] = u;
    depth[v] = depth[u] + 1;
    dfs_size(v);
    sz[u] += sz[v];
    if (heavy[u] == -1 || sz[v] > sz[heavy[u]]) heavy[u] = v;
  }
}
void dfs_hld(int u, int c) {
  head[u] = c;
  pos[u] = timer++;
  if (heavy[u] != -1) dfs_hld(heavy[u], c);
  for (int v : adj[u])
    if (v != parent[u] && v != heavy[u]) dfs_hld(v, v);
}
void build(int root = 0) {
  parent[root] = root;
  depth[root] = 0;
  dfs_size(root);
  dfs_hld(root, root);
}
int lca(int u, int v) {
  while (head[u] != head[v]) {
    if (depth[head[u]] < depth[head[v]]) swap(u, v);
    u = parent[head[u]];
  }
  return depth[u] < depth[v] ? u : v;
}
int dist(int u, int v) {
  return depth[u] + depth[v] - 2 * depth[lca(u, v)];
}
// Llama f(l, r) para cada tramo [l, r] del camino u-v.
template <class F>
void path(int u, int v, F f) {
  while (head[u] != head[v]) {
    if (depth[head[u]] < depth[head[v]]) swap(u, v);
    f(pos[head[u]], pos[u]);
    u = parent[head[u]];
  }
  if (depth[u] > depth[v]) swap(u, v);
  f(pos[u], pos[v]);
}
void update_path(int u, int v, long long x) {
  path(u, v, [&](int l, int r) { st.update(l, r, x); });
}
long long query_path(int u, int v) {
  long long s = 0;
  path(u, v, [&](int l, int r) { s += st.query(l, r); });
  return s;
}
void update_subtree(int u, long long x) {
  st.update(pos[u], pos[u] + sz[u] - 1, x);
}
long long query_subtree(int u) {
  return st.query(pos[u], pos[u] + sz[u] - 1);
}
};
```

2.11 Centroid Decomposition

```
// Centroid Decomposition. build() arma el arbol de centroides
// (altura O(log n)) en O(n log n).
// parent[c]: padre de c en el arbol de centroides (-1 la raiz).
// dist[k][v]: distancia de v a su ancestro centroide de nivel k.
// Ejemplo incluido: mark(v) y closest(v) = distancia al
// vertice marcado mas cercano, ambas en O(log n).
struct Centroid {
  int n;
  vector<vector<int>> adj, dist;
  vector<int> sz, parent, level, best;
  vector<bool> removed;
  Centroid(int n_) : n(n_), adj(n_), sz(n_), parent(n_, -1),
      level(n_, 0), best(n_, INT_MAX / 2), removed(n_, false) {}
  void add_edge(int u, int v) {
    adj[u].push_back(v);
    adj[v].push_back(u);
  }
  int calc_size(int u, int p) {
    sz[u] = 1;
    for (int v : adj[u])
      if (v != p && !removed[v]) sz[u] += calc_size(v, u);
    return sz[u];
  }
  int find_centroid(int u, int p, int total) {
    for (int v : adj[u])
      if (v != p && !removed[v] && 2 * sz[v] > total)
        return find_centroid(v, u, total);
    return u;
  }
  void calc_dist(int u, int p, int k, int d) {
    dist[k][u] = d;
    for (int v : adj[u])
      if (v != p && !removed[v]) calc_dist(v, u, k, d + 1);
  }
  void build(int u = 0, int p = -1, int k = 0) {
    int c = find_centroid(u, -1, calc_size(u, -1));
    parent[c] = p;
    level[c] = k;
    if ((int)dist.size() <= k) dist.emplace_back(n, 0);
    calc_dist(c, -1, k, 0);
    // Aqui: procesar los caminos que pasan por c
    // (DFS desde c sin cruzar vertices quitados).
    removed[c] = true;
    for (int v : adj[c])
      if (!removed[v]) build(v, c, k + 1);
  }
  void mark(int v) {
    for (int c = v; c != -1; c = parent[c])
      best[c] = min(best[c], dist[level[c]][v]);
  }
```

```
  int closest(int v) {  // >= INT_MAX / 2 si no hay marcados
    int res = INT_MAX / 2;
    for (int c = v; c != -1; c = parent[c])
      res = min(res, best[c] + dist[level[c]][v]);
    return res;
  }
};
```

2.12 Ordered Set (pb_ds)

```
// Set con estadisticas de orden (GCC). O(log n).
// s.find_by_order(k): iterador al k-esimo menor (desde 0).
// s.order_of_key(x): cantidad de elementos menores que x.
// Para multiset: guardar pair<valor, id unico>.
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
template <class T>
using ordered_set = __gnu_pbds::tree<
    T, __gnu_pbds::null_type, less<T>, __gnu_pbds::rb_tree_tag,
    __gnu_pbds::tree_order_statistics_node_update>;
```

2.13 Li Chao Tree

```
// Li Chao Tree dinamico: minimo de rectas y = m x + b evaluadas
// en x entero de [lo, hi]. add y query en O(log(hi - lo)).
// Sirve para Convex Hull Trick sin exigir pendientes ordenadas.
// Para maximo: insertar (-m, -b) y negar la respuesta.
// Cuidado: m * x + b debe caber en long long.
struct LiChao {
  struct Line {
    long long m, b;
    long long eval(long long x) const { return m * x + b; }
  };
  struct Node { Line r; int lc, rc; };
  long long lo, hi;
  vector<Node> t;
  LiChao(long long lo_, long long hi_) : lo(lo_), hi(hi_) {
    new_node();
  }
  int new_node() {
    t.push_back({{0, LLONG_MAX}, -1, -1});
    return (int)t.size() - 1;
  }
  void add(Line added, int v, long long l, long long r) {
    long long mid = l + (r - l) / 2;
    bool wins_left = added.eval(l) < t[v].r.eval(l);
    bool wins_mid = added.eval(mid) < t[v].r.eval(mid);
    if (wins_mid) swap(t[v].r, added);
    if (l == r || added.b == LLONG_MAX) return;
    if (wins_left != wins_mid) {
      if (t[v].lc == -1) { int id = new_node(); t[v].lc = id; }
      add(added, t[v].lc, l, mid);
    } else {
      if (t[v].rc == -1) { int id = new_node(); t[v].rc = id; }
      add(added, t[v].rc, mid + 1, r);
    }
  }
  void add(long long m, long long b) { add({m, b}, 0, lo, hi); }
  long long query(long long x) {  // LLONG_MAX si no hay rectas
    long long res = LLONG_MAX, l = lo, r = hi;
    int v = 0;
    while (v != -1) {
      res = min(res, t[v].r.eval(x));
      long long mid = l + (r - l) / 2;
      if (x <= mid) { v = t[v].lc; r = mid; }
      else { v = t[v].rc; l = mid + 1; }
    }
    return res;
  }
};
```

3. Grafos

3.1 Dijkstra

```
// Dijkstra con cola de prioridad. O((n + m) log n).
// adj[u] = lista de (vecino, peso >= 0). LLONG_MAX = inalcanzable.
vector<long long> dijkstra(
    const vector<vector<pair<int, long long>>>& adj, int s) {
  vector<long long> dist(adj.size(), LLONG_MAX);
  priority_queue<pair<long long, int>,
                 vector<pair<long long, int>>,
                 greater<pair<long long, int>>> pq;
  dist[s] = 0;
  pq.push({0, s});
  while (!pq.empty()) {
    auto [d, u] = pq.top();
    pq.pop();
    if (d > dist[u]) continue;
    for (auto [v, w] : adj[u])
      if (d + w < dist[v]) {
        dist[v] = d + w;
        pq.push({dist[v], v});
      }
  }
  return dist;
}
```

3.2 Biconexas, puentes, articulación, bridge tree

```
// Puentes, puntos de articulacion, componentes biconexas y
// bridge tree. Grafo no dirigido; soporta aristas multiples.
// Uso: add_edge(...), run(), y opcionalmente bridge_tree().
// O(n + m). Requiere C++17.
struct Biconnected {
  int n, m = 0, timer = 0;
  vector<vector<pair<int, int>>> adj;  // (vecino, id de arista)
  vector<pair<int, int>> edges;
  vector<int> tin, low, stk;
  vector<bool> is_bridge, is_articulation;
  vector<vector<int>> comps;  // ids de aristas de cada biconexa
  Biconnected(int n_) : n(n_), adj(n_), tin(n_, -1), low(n_, 0),
                        is_articulation(n_, false) {}
  void add_edge(int u, int v) {
```

```
    adj[u].push_back({v, m});
    adj[v].push_back({u, m});
    edges.push_back({u, v});
    m++;
  }
  void dfs(int u, int parent_edge) {
    tin[u] = low[u] = timer++;
    int children = 0;
    for (auto [v, id] : adj[u]) {
      if (id == parent_edge) continue;
      if (tin[v] != -1) {  // arista de retroceso
        low[u] = min(low[u], tin[v]);
        if (tin[v] < tin[u]) stk.push_back(id);
        continue;
      }
      stk.push_back(id);
      dfs(v, id);
      children++;
      low[u] = min(low[u], low[v]);
      if (low[v] > tin[u]) is_bridge[id] = true;
      if (low[v] >= tin[u]) {
        if (parent_edge != -1 || children > 1) is_articulation[u] = true;
        comps.emplace_back();
        while (true) {
          int e = stk.back();
          stk.pop_back();
          comps.back().push_back(e);
          if (e == id) break;
        }
      }
    }
  }
  void run() {
    is_bridge.assign(m, false);
    for (int u = 0; u < n; u++)
      if (tin[u] == -1) dfs(u, -1);
  }
  // Bridge tree: comp[u] = componente 2-arista-conexa de u.
  // tree = bosque cuyas aristas son los puentes. Retorna #comps.
  vector<int> comp;
  vector<vector<int>> tree;
  int bridge_tree() {
    comp.assign(n, -1);
    int c = 0;
    for (int s = 0; s < n; s++) {
      if (comp[s] != -1) continue;
      vector<int> pending = {s};
      comp[s] = c;
      while (!pending.empty()) {
        int u = pending.back();
        pending.pop_back();
        for (auto [v, id] : adj[u])
          if (!is_bridge[id] && comp[v] == -1) {
            comp[v] = c;
            pending.push_back(v);
          }
      }
      c++;
    }
    tree.assign(c, {});
    for (int id = 0; id < m; id++)
      if (is_bridge[id]) {
        auto [u, v] = edges[id];
        tree[comp[u]].push_back(comp[v]);
        tree[comp[v]].push_back(comp[u]);
      }
    return c;
  }
};
```

3.3 Camino y circuito euleriano (Hierholzer)

```
// Camino / circuito euleriano (Hierholzer iterativo). O(n + m).
// Recorre cada ARISTA exactamente una vez. Admite aristas
// multiples y lazos, dirigido o no dirigido.
// path(): secuencia de m + 1 vertices, o vacio si no existe.
//   Si el primero y el ultimo coinciden es un circuito (se puede
//   rotar para empezar en cualquier vertice).
// Condiciones de existencia (todas las aristas en una componente):
//   No dirigido: 0 vertices de grado impar (circuito) o 2 (camino
//     que va de uno al otro).
//   Dirigido: todos con entrada == salida (circuito), o uno con
//     salida - entrada = 1 (inicio) y otro con entrada - salida = 1
//     (final) y el resto iguales.
struct Euler {
  int n, m = 0;
  bool directed;
  vector<vector<pair<int, int>>> adj;  // (vecino, id de arista)
  vector<int> out_deg, in_deg;
  Euler(int n_, bool directed_) : n(n_), directed(directed_),
      adj(n_), out_deg(n_, 0), in_deg(n_, 0) {}
  void add_edge(int u, int v) {
    adj[u].push_back({v, m});
    if (!directed) adj[v].push_back({u, m});
    out_deg[u]++;
    in_deg[v]++;
    m++;
  }
  vector<int> path() {
    int start = -1, odd_cnt = 0, ends = 0;
    for (int u = 0; u < n; u++) {
      if (directed) {
        int d = out_deg[u] - in_deg[u];
        if (d == 1) { start = u; ends++; }
        else if (d == -1) ends++;
        else if (d != 0) return {};
      } else if ((out_deg[u] + in_deg[u]) % 2 == 1) {
        odd_cnt++;
        if (start == -1) start = u;
      }
```

```
    }
    if (directed && ends != 0 && !(ends == 2 && start != -1))
      return {};
    if (!directed && odd_cnt != 0 && odd_cnt != 2) return {};
    if (start == -1)  // circuito: cualquier vertice con aristas
      for (int u = 0; u < n && start == -1; u++)
        if (!adj[u].empty()) start = u;
    if (start == -1) return {0};  // grafo sin aristas
    vector<int> it(n, 0), res, stk = {start};
    vector<bool> used(m, false);
    while (!stk.empty()) {
      int u = stk.back();
      while (it[u] < (int)adj[u].size() && used[adj[u][it[u]].second])
        it[u]++;
      if (it[u] == (int)adj[u].size()) {
        res.push_back(u);  // u ya no tiene aristas libres
        stk.pop_back();
      } else {
        auto [v, id] = adj[u][it[u]];
        used[id] = true;
        stk.push_back(v);
      }
    }
    if ((int)res.size() != m + 1) return {};  // aristas desconectadas
    reverse(res.begin(), res.end());
    return res;
  }
};
```

3.4 SCC y grafo reducido

```
// Componentes fuertemente conexas (Tarjan) + grafo reducido.
// run() retorna la cantidad de componentes. comp[u] queda en
// orden topologico: si hay arista c1 -> c2 en el DAG, c1 < c2.
// O(n + m).
struct SCC {
  int n, timer = 0, ncomp = 0;
  vector<vector<int>> adj;
  vector<int> tin, low, comp, stk;
  SCC(int n_) : n(n_), adj(n_), tin(n_, -1), low(n_, 0),
                comp(n_, -1) {}
  void add_edge(int u, int v) { adj[u].push_back(v); }
  void dfs(int u) {
    tin[u] = low[u] = timer++;
    stk.push_back(u);
    for (int v : adj[u]) {
      if (tin[v] == -1) {
        dfs(v);
        low[u] = min(low[u], low[v]);
      } else if (comp[v] == -1) {
        low[u] = min(low[u], tin[v]);
      }
    }
    if (low[u] == tin[u]) {
      while (true) {
        int v = stk.back();
        stk.pop_back();
        comp[v] = ncomp;
        if (v == u) break;
      }
      ncomp++;
    }
  }
  int run() {
    for (int u = 0; u < n; u++)
      if (tin[u] == -1) dfs(u);
    for (int& c : comp) c = ncomp - 1 - c;
    return ncomp;
  }
  // Grafo reducido (DAG de componentes), sin aristas repetidas.
  vector<vector<int>> condense() {
    vector<vector<int>> dag(ncomp);
    for (int u = 0; u < n; u++)
      for (int v : adj[u])
        if (comp[u] != comp[v]) dag[comp[u]].push_back(comp[v]);
    for (auto& row : dag) {
      sort(row.begin(), row.end());
      row.erase(unique(row.begin(), row.end()), row.end());
    }
    return dag;
  }
};
```

3.5 2-SAT

```
// 2-SAT. Requiere SCC. Variables 0..n-1. O(n + clausulas).
// add_or(a, va, b, vb): agrega la clausula
//   (x_a == va) OR (x_b == vb).
// solve(): false si es insatisfacible; si no, value[i] es una
// asignacion valida.
struct TwoSat {
  int n;
  SCC scc;
  vector<bool> value;
  TwoSat(int n_) : n(n_), scc(2 * n_), value(n_, false) {}
  // nodo 2a = "x_a verdadero", nodo 2a+1 = "x_a falso"
  void add_or(int a, bool va, int b, bool vb) {
    int x = 2 * a + (va ? 0 : 1), y = 2 * b + (vb ? 0 : 1);
    scc.add_edge(x ^ 1, y);
    scc.add_edge(y ^ 1, x);
  }
  void force(int a, bool va) { add_or(a, va, a, va); }
  void implies(int a, bool va, int b, bool vb) {
    add_or(a, !va, b, vb);
  }
  void distinct(int a, int b) {  // x_a XOR x_b
    add_or(a, true, b, true);
    add_or(a, false, b, false);
  }
  bool solve() {
    scc.run();
    for (int i = 0; i < n; i++) {
```

```
      if (scc.comp[2 * i] == scc.comp[2 * i + 1]) return false;
      value[i] = scc.comp[2 * i] > scc.comp[2 * i + 1];
    }
    return true;
  }
};
```

3.6 Flujo máximo (Dinic)

```
// Flujo maximo (Dinic). O(V^2 E); O(E sqrt V) en grafos unitarios
// (matching bipartito). En la practica es muy rapido.
// add_edge(u, v, cap, rcap): rcap = cap de vuelta (no dirigido).
// Tras max_flow: level[u] != -1 <=> u queda del lado de s en el
// corte minimo. Flujo por la arista i-esima: e[2*i+1].cap
// (solo si se agrego con rcap = 0).
struct Dinic {
  struct Edge { int to; long long cap; };
  int n;
  vector<Edge> e;
  vector<vector<int>> adj;
  vector<int> level, it;
  Dinic(int n_) : n(n_), adj(n_), level(n_), it(n_) {}
  void add_edge(int u, int v, long long cap, long long rcap = 0) {
    adj[u].push_back(e.size());
    e.push_back({v, cap});
    adj[v].push_back(e.size());
    e.push_back({u, rcap});
  }
  bool bfs(int s, int t) {
    fill(level.begin(), level.end(), -1);
    level[s] = 0;
    queue<int> q;
    q.push(s);
    while (!q.empty()) {
      int u = q.front();
      q.pop();
      for (int id : adj[u]) {
        int v = e[id].to;
        if (e[id].cap > 0 && level[v] == -1) {
          level[v] = level[u] + 1;
          q.push(v);
        }
      }
    }
    return level[t] != -1;
  }
  long long dfs(int u, int t, long long f) {
    if (u == t) return f;
    for (int& i = it[u]; i < (int)adj[u].size(); i++) {
      int id = adj[u][i], v = e[id].to;
      if (e[id].cap == 0 || level[v] != level[u] + 1) continue;
      long long d = dfs(v, t, min(f, e[id].cap));
      if (d > 0) {
        e[id].cap -= d;
        e[id ^ 1].cap += d;
        return d;
      }
    }
    return 0;
  }
  long long max_flow(int s, int t) {
    long long flow = 0;
    while (bfs(s, t)) {
      fill(it.begin(), it.end(), 0);
      while (long long f = dfs(s, t, LLONG_MAX)) flow += f;
    }
    return flow;
  }
};
```

3.7 Flujo máximo de costo mínimo

```
// Flujo maximo de costo minimo (caminos de aumento con SPFA).
// O(F * V * E) en el peor caso; suficiente en la practica.
// Admite costos negativos mientras no haya ciclos negativos.
// flow(s, t, limit) retorna (flujo enviado, costo total).
struct MinCostFlow {
  struct Edge { int to; long long cap, cost; };
  int n;
  vector<Edge> e;
  vector<vector<int>> adj;
  vector<long long> dist;
  vector<int> prev_edge;
  vector<bool> in_queue;
  MinCostFlow(int n_) : n(n_), adj(n_), dist(n_), prev_edge(n_),
                        in_queue(n_) {}
  void add_edge(int u, int v, long long cap, long long cost) {
    adj[u].push_back(e.size());
    e.push_back({v, cap, cost});
    adj[v].push_back(e.size());
    e.push_back({u, 0, -cost});
  }
  bool spfa(int s, int t) {
    fill(dist.begin(), dist.end(), LLONG_MAX);
    fill(in_queue.begin(), in_queue.end(), false);
    queue<int> q;
    dist[s] = 0;
    q.push(s);
    while (!q.empty()) {
      int u = q.front();
      q.pop();
      in_queue[u] = false;
      for (int id : adj[u]) {
        int v = e[id].to;
        if (e[id].cap > 0 && dist[u] + e[id].cost < dist[v]) {
          dist[v] = dist[u] + e[id].cost;
          prev_edge[v] = id;
          if (!in_queue[v]) {
            in_queue[v] = true;
            q.push(v);
          }
        }
```

```
      }
    }
    return dist[t] != LLONG_MAX;
  }
  pair<long long, long long> flow(int s, int t,
                                  long long limit = LLONG_MAX) {
    long long f = 0, c = 0;
    while (f < limit && spfa(s, t)) {
      long long aug = limit - f;
      for (int v = t; v != s; v = e[prev_edge[v] ^ 1].to)
        aug = min(aug, e[prev_edge[v]].cap);
      for (int v = t; v != s; v = e[prev_edge[v] ^ 1].to) {
        e[prev_edge[v]].cap -= aug;
        e[prev_edge[v] ^ 1].cap += aug;
      }
      f += aug;
      c += aug * dist[t];
    }
    return {f, c};
  }
};
```

3.8 Húngaro (asignación de peso mín/máx)

```
// Algoritmo hungaro: asignacion de costo MINIMO en O(n^2 m).
// a[i][j] = costo de asignar la fila i a la columna j, con n <= m.
// Retorna (cost, assignment) con assignment[i] = columna de la fila i.
// Para MAXIMO peso: negar los costos y negar el resultado.
pair<long long, vector<int>> hungarian(
    const vector<vector<long long>>& a) {
  int n = a.size(), m = a[0].size();
  const long long INF = LLONG_MAX / 4;
  vector<long long> u(n + 1, 0), v(m + 1, 0);
  vector<int> p(m + 1, 0), path(m + 1, 0);
  for (int i = 1; i <= n; i++) {
    p[0] = i;
    int j0 = 0;
    vector<long long> minv(m + 1, INF);
    vector<bool> used(m + 1, false);
    do {
      used[j0] = true;
      int i0 = p[j0], j1 = 0;
      long long delta = INF;
      for (int j = 1; j <= m; j++) {
        if (used[j]) continue;
        long long cur = a[i0 - 1][j - 1] - u[i0] - v[j];
        if (cur < minv[j]) { minv[j] = cur; path[j] = j0; }
        if (minv[j] < delta) { delta = minv[j]; j1 = j; }
      }
      for (int j = 0; j <= m; j++) {
        if (used[j]) { u[p[j]] += delta; v[j] -= delta; }
        else minv[j] -= delta;
      }
      j0 = j1;
    } while (p[j0] != 0);
    do {
      int j1 = path[j0];
      p[j0] = p[j1];
      j0 = j1;
    } while (j0 != 0);
  }
  vector<int> assignment(n, -1);
  for (int j = 1; j <= m; j++)
    if (p[j] != 0) assignment[p[j] - 1] = j - 1;
  return {-v[0], assignment};
}
```

3.9 Matching bipartito (Hopcroft-Karp)

```
// Matching bipartito maximo (Hopcroft-Karp). O(E sqrt V).
// Izquierda 0..n-1, derecha 0..m-1. add_edge(u izq, v der).
// Konig: cobertura minima de vertices = matching maximo;
// conjunto independiente maximo = n + m - matching.
struct HopcroftKarp {
  int n, m;
  vector<vector<int>> adj;
  vector<int> match_left, match_right, dist;
  HopcroftKarp(int n_, int m_) : n(n_), m(m_), adj(n_),
      match_left(n_, -1), match_right(m_, -1), dist(n_) {}
  void add_edge(int u, int v) { adj[u].push_back(v); }
  bool bfs() {
    queue<int> q;
    bool found_path = false;
    for (int u = 0; u < n; u++) {
      dist[u] = match_left[u] == -1 ? 0 : -1;
      if (dist[u] == 0) q.push(u);
    }
    while (!q.empty()) {
      int u = q.front();
      q.pop();
      for (int v : adj[u]) {
        int w = match_right[v];
        if (w == -1) found_path = true;
        else if (dist[w] == -1) {
          dist[w] = dist[u] + 1;
          q.push(w);
        }
      }
    }
    return found_path;
  }
  bool dfs(int u) {
    for (int v : adj[u]) {
      int w = match_right[v];
      if (w == -1 || (dist[w] == dist[u] + 1 && dfs(w))) {
        match_left[u] = v;
        match_right[v] = u;
        return true;
      }
    }
    dist[u] = -1;
    return false;
```

```
  }
  int matching() {
    int res = 0;
    while (bfs())
      for (int u = 0; u < n; u++)
        if (match_left[u] == -1 && dfs(u)) res++;
    return res;
  }
};
```

4. Matemática

4.1 Aritmética modular

```
// Aritmetica modular de 64 bits (usa __int128 de GCC).
// Requiere 0 <= a, b < m en mulmod. powmod en O(log e).
// Inverso con m primo: powmod(a, m - 2, m).
long long mulmod(long long a, long long b, long long m) {
  return (long long)((__int128)a * b % m);
}
long long powmod(long long b, long long e, long long m) {
  long long r = 1 % m;
  b %= m;
  while (e > 0) {
    if (e & 1) r = mulmod(r, b, m);
    b = mulmod(b, b, m);
    e >>= 1;
  }
  return r;
}
```

4.2 Miller-Rabin

```
// Miller-Rabin determinista para todo n < 2^63.
// Requiere mulmod y powmod. O(12 log n) multiplicaciones.
bool is_prime(long long n) {
  if (n < 2) return false;
  for (long long p : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37})
    if (n % p == 0) return n == p;
  long long d = n - 1;
  int s = 0;
  while (d % 2 == 0) {
    d /= 2;
    s++;
  }
  for (long long a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) {
    long long x = powmod(a, d, n);
    if (x == 1 || x == n - 1) continue;
    bool composite = true;
    for (int r = 1; r < s && composite; r++) {
      x = mulmod(x, x, n);
      if (x == n - 1) composite = false;
    }
    if (composite) return false;
  }
  return true;
}
```

4.3 Pollard Rho

```
// Pollard Rho: factorizacion de n < 2^63 en ~O(n^(1/4)).
// Requiere mulmod y is_prime (Miller-Rabin).
// rho(n): un factor no trivial de n (n compuesto).
// factorize(n): factores primos con repeticion, ordenados.
long long rho(long long n) {
  if (n % 2 == 0) return 2;
  static mt19937_64 gen(20260101);
  while (true) {
    long long c = gen() % (n - 1) + 1;
    long long x = gen() % n, y = x, q = 1;
    auto f = [&](long long v) { return (mulmod(v, v, n) + c) % n; };
    for (int it = 1;; it++) {
      x = f(x);
      y = f(f(y));
      long long p = mulmod(q, llabs(x - y), n);
      if (p != 0) q = p;  // q nunca se vuelve multiplo de n
      if (p == 0 || it % 32 == 0) {
        long long d = __gcd(q, n);
        if (d > 1) return d;
        if (x == y) break;  // ciclo sin exito: otra constante
      }
    }
  }
}
void factorize_rec(long long n, vector<long long>& fs) {
  if (n == 1) return;
  if (is_prime(n)) { fs.push_back(n); return; }
  long long d = rho(n);
  factorize_rec(d, fs);
  factorize_rec(n / d, fs);
}
vector<long long> factorize(long long n) {
  vector<long long> fs;
  factorize_rec(n, fs);
  sort(fs.begin(), fs.end());
  return fs;
}
```

4.4 Euclides extendido, inverso, diofántica

```
// Euclides extendido: retorna g = gcd(a, b) y deja x, y con
// a x + b y = g. O(log min(a, b)). Para a, b >= 0.
long long extgcd(long long a, long long b, long long& x,
                 long long& y) {
  if (b == 0) { x = 1; y = 0; return a; }
  long long x1, y1;
  long long g = extgcd(b, a % b, x1, y1);
  x = y1;
  y = x1 - (a / b) * y1;
  return g;
}
// Inverso modular de a modulo m (m no necesita ser primo).
// Retorna -1 si no existe (gcd(a, m) != 1).
long long inv_mod(long long a, long long m) {
```

```
  long long x, y;
  long long g = extgcd(((a % m) + m) % m, m, x, y);
  return g == 1 ? ((x % m) + m) % m : -1;
}
// Ecuacion diofantica a x + b y = c. Retorna false si no hay
// solucion. Si hay, (x0, y0) es una y todas las demas son
//   x = x0 + k * (b / g),  y = y0 - k * (a / g),  k entero.
bool diophantine(long long a, long long b, long long c,
                 long long& x0, long long& y0, long long& g) {
  g = extgcd(llabs(a), llabs(b), x0, y0);
  if (g == 0) { x0 = y0 = 0; return c == 0; }
  if (c % g != 0) return false;
  x0 *= c / g;
  y0 *= c / g;
  if (a < 0) x0 = -x0;
  if (b < 0) y0 = -y0;
  return true;
}
```

4.5 Teorema chino del resto

```
// Teorema chino del resto generalizado (modulos no coprimos).
// Requiere extgcd. Une x = a1 (mod m1) con x = a2 (mod m2).
// Retorna (x, lcm(m1, m2)) con 0 <= x < lcm, o (-1, -1) si no
// hay solucion. El lcm debe caber en long long.
// Para varias congruencias: plegar empezando con (0, 1).
pair<long long, long long> crt(long long a1, long long m1,
                               long long a2, long long m2) {
  long long p, q;
  long long g = extgcd(m1, m2, p, q);
  if ((a2 - a1) % g != 0) return {-1, -1};
  long long l = m1 / g * m2, mg = m2 / g;
  __int128 diff = (a2 - a1) / g % mg;
  long long k = (long long)(diff * p % mg);
  long long x = (long long)((a1 + (__int128)k * m1) % l);
  return {(x + l) % l, l};
}
```

4.6 Baby Step / Giant Step

```
// Logaritmo discreto (Baby Step / Giant Step). O(sqrt m).
// Requiere mulmod y powmod. Retorna el menor x >= 0 tal que
// a^x = b (mod m), o -1 si no existe. No exige gcd(a, m) = 1.
// Convencion: 0^0 = 1.
long long discrete_log(long long a, long long b, long long m) {
  a %= m;
  b %= m;
  long long k = 1 % m, offset = 0, g;
  while ((g = __gcd(a, m)) > 1) {  // quitar factores comunes
    if (b == k) return offset;
    if (b % g != 0) return -1;
    b /= g;
    m /= g;
    offset++;
    k = mulmod(k, a / g, m);
  }
  long long n = (long long)sqrtl((long double)m) + 1;
  unordered_map<long long, long long> table;  // b * a^q -> q
  long long cur = b % m;
  for (long long q = 0; q <= n; q++) {
    table[cur] = q;
    cur = mulmod(cur, a % m, m);
  }
  long long an = powmod(a, n, m);
  cur = k % m;
  for (long long p = 1; p <= n; p++) {
    cur = mulmod(cur, an, m);
    auto it = table.find(cur);
    if (it != table.end()) return n * p - it->second + offset;
  }
  return -1;
}
```

4.7 Criba lineal

```
// Criba lineal en O(n). lp[x] = menor primo que divide a x.
// factorize(x) en O(log x) para x <= n.
struct Sieve {
  vector<int> lp, primes;
  Sieve(int n) : lp(n + 1, 0) {
    for (int i = 2; i <= n; i++) {
      if (lp[i] == 0) {
        lp[i] = i;
        primes.push_back(i);
      }
      for (int p : primes) {
        if (p > lp[i] || (long long)i * p > n) break;
        lp[i * p] = p;
      }
    }
  }
  bool is_prime(int x) { return x >= 2 && lp[x] == x; }
  vector<pair<int, int>> factorize(int x) {  // (primo, exponente)
    vector<pair<int, int>> f;
    while (x > 1) {
      int p = lp[x], e = 0;
      while (x % p == 0) {
        x /= p;
        e++;
      }
      f.push_back({p, e});
    }
    return f;
  }
};
```

4.8 Gauss (reales y módulo primo)

```
// Eliminacion de Gauss-Jordan. O(n m min(n, m)).
// a = matriz aumentada de n filas y m + 1 columnas (A | b).
// Retorna 0 (sin solucion), 1 (unica) o 2 (infinitas).
// ans = una solucion (variables libres en 0).
// Cada fila se escala por su mayor valor absoluto: asi EPS es
// relativo y funciona igual con coeficientes chicos o grandes.
```

```cpp
int gauss(vector<vector<double>> a, vector<double>& ans) {
  const double EPS = 1e-9;
  int n = a.size(), m = a[0].size() - 1, rank = 0;
  for (auto& r : a) {
    double mx = 0;
    for (double v : r) mx = max(mx, fabs(v));
    if (mx > 0)
      for (double& v : r) v /= mx;
  }
  vector<int> where(m, -1);
  for (int col = 0; col < m && rank < n; col++) {
    int piv = rank;
    for (int i = rank; i < n; i++)
      if (fabs(a[i][col]) > fabs(a[piv][col])) piv = i;
    if (fabs(a[piv][col]) < EPS) continue;
    swap(a[piv], a[rank]);
    where[col] = rank;
    for (int i = 0; i < n; i++) {
      if (i == rank) continue;
      double c = a[i][col] / a[rank][col];
      for (int j = col; j <= m; j++) a[i][j] -= a[rank][j] * c;
    }
    rank++;
  }
  ans.assign(m, 0);
  for (int j = 0; j < m; j++)
    if (where[j] != -1) ans[j] = a[where[j]][m] / a[where[j]][j];
  for (int i = rank; i < n; i++)  // filas sin pivote: quedan 0 = b
    if (fabs(a[i][m]) > EPS) return 0;
  for (int j = 0; j < m; j++)
    if (where[j] == -1) return 2;
  return 1;
}

// Igual pero modulo un primo. Requiere powmod.
// Entradas en [0, mod). El rango es la cantidad de where[j] != -1.
int gauss_mod(vector<vector<long long>> a, vector<long long>& ans,
              long long mod) {
  int n = a.size(), m = a[0].size() - 1;
  vector<int> where(m, -1);
  for (int col = 0, row = 0; col < m && row < n; col++) {
    int piv = row;
    while (piv < n && a[piv][col] == 0) piv++;
    if (piv == n) continue;
    swap(a[piv], a[row]);
    where[col] = row;
    long long inv = powmod(a[row][col], mod - 2, mod);
    for (int j = col; j <= m; j++)
      a[row][j] = a[row][j] * inv % mod;
    for (int i = 0; i < n; i++) {
      if (i == row || a[i][col] == 0) continue;
      long long c = a[i][col];
      for (int j = col; j <= m; j++)
        a[i][j] = ((a[i][j] - a[row][j] * c) % mod + mod) % mod;
    }
    row++;
  }
  ans.assign(m, 0);
  for (int j = 0; j < m; j++)
    if (where[j] != -1) ans[j] = a[where[j]][m];
  for (int i = 0; i < n; i++) {
    long long s = 0;
    for (int j = 0; j < m; j++) s = (s + ans[j] * a[i][j]) % mod;
    if (s != a[i][m]) return 0;
  }
  for (int j = 0; j < m; j++)
    if (where[j] == -1) return 2;
  return 1;
}
```

4.9 FFT

```cpp
// FFT iterativa con complex<double>. O(n log n).
// multiply(a, b): producto de polinomios con coeficientes
// enteros. Exacto mientras los coeficientes del resultado no
// pasen de ~1e14 (probado: n = 2e4 con valores hasta 1e5).
// Mas alla de eso, o modulo 998244353, usar NTT.
void fft(vector<complex<double>>& a, bool invert) {
  const double PI = acos(-1.0);
  int n = a.size();
  for (int i = 1, j = 0; i < n; i++) {
    int bit = n >> 1;
    for (; j & bit; bit >>= 1) j ^= bit;
    j ^= bit;
    if (i < j) swap(a[i], a[j]);
  }
  vector<complex<double>> root(n / 2);
  for (int i = 0; i < n / 2; i++)
    root[i] = polar(1.0, 2 * PI * i / n * (invert ? -1 : 1));
  for (int len = 2; len <= n; len *= 2)
    for (int i = 0; i < n; i += len)
      for (int j = 0; j < len / 2; j++) {
        complex<double> u = a[i + j];
        complex<double> v = a[i + j + len / 2] * root[n / len * j];
        a[i + j] = u + v;
        a[i + j + len / 2] = u - v;
      }
  if (invert)
    for (complex<double>& x : a) x /= n;
}
vector<long long> multiply(const vector<long long>& a,
                           const vector<long long>& b) {
  if (a.empty() || b.empty()) return {};
  int sz = a.size() + b.size() - 1, n = 1;
  while (n < sz) n *= 2;
  vector<complex<double>> fa(n), fb(n);
  for (int i = 0; i < (int)a.size(); i++) fa[i] = (double)a[i];
  for (int i = 0; i < (int)b.size(); i++) fb[i] = (double)b[i];
  fft(fa, false);
  fft(fb, false);
```

```
  for (int i = 0; i < n; i++) fa[i] *= fb[i];
  fft(fa, true);
  vector<long long> res(sz);
  for (int i = 0; i < sz; i++) res[i] = llround(fa[i].real());
  return res;
}
```

4.10 NTT

```
// NTT modulo 998244353 = 119 * 2^23 + 1 (raiz primitiva 3).
// Requiere powmod. Exacta, O(n log n), n hasta 2^23.
// Otros primos validos: 469762049 y 167772161 (ambos raiz 3).
const long long MOD = 998244353, ROOT = 3;

void ntt(vector<long long>& a, bool invert) {
  int n = a.size();
  for (int i = 1, j = 0; i < n; i++) {
    int bit = n >> 1;
    for (; j & bit; bit >>= 1) j ^= bit;
    j ^= bit;
    if (i < j) swap(a[i], a[j]);
  }
  for (int len = 2; len <= n; len *= 2) {
    long long wlen = powmod(ROOT, (MOD - 1) / len, MOD);
    if (invert) wlen = powmod(wlen, MOD - 2, MOD);
    for (int i = 0; i < n; i += len) {
      long long w = 1;
      for (int j = 0; j < len / 2; j++) {
        long long u = a[i + j];
        long long v = a[i + j + len / 2] * w % MOD;
        a[i + j] = (u + v) % MOD;
        a[i + j + len / 2] = (u - v + MOD) % MOD;
        w = w * wlen % MOD;
      }
    }
  }
  if (invert) {
    long long inv_n = powmod(n, MOD - 2, MOD);
    for (long long& x : a) x = x * inv_n % MOD;
  }
}
// Producto de polinomios con coeficientes en [0, MOD).
vector<long long> multiply_mod(vector<long long> a,
                               vector<long long> b) {
  if (a.empty() || b.empty()) return {};
  int sz = a.size() + b.size() - 1, n = 1;
  while (n < sz) n *= 2;
  a.resize(n);
  b.resize(n);
  ntt(a, false);
  ntt(b, false);
  for (int i = 0; i < n; i++) a[i] = a[i] * b[i] % MOD;
  ntt(a, true);
  a.resize(sz);
  return a;
}
```

4.11 Matrices

```
// Multiplicacion y exponenciacion de matrices cuadradas modulo
// mod. O(k^3 log e). Sirve para recurrencias lineales.
// Requiere mod < 3e9 para que no desborde el producto.
vector<vector<long long>> mat_mult(
    const vector<vector<long long>>& a,
    const vector<vector<long long>>& b, long long mod) {
  int k = a.size();
  vector<vector<long long>> c(k, vector<long long>(k, 0));
  for (int i = 0; i < k; i++)
    for (int l = 0; l < k; l++) {
      if (a[i][l] == 0) continue;
      for (int j = 0; j < k; j++)
        c[i][j] = (c[i][j] + a[i][l] * b[l][j]) % mod;
    }
  return c;
}
vector<vector<long long>> mat_pow(vector<vector<long long>> a,
                                  long long e, long long mod) {
  int k = a.size();
  vector<vector<long long>> r(k, vector<long long>(k, 0));
  for (int i = 0; i < k; i++) r[i][i] = 1 % mod;
  while (e > 0) {
    if (e & 1) r = mat_mult(r, a, mod);
    a = mat_mult(a, a, mod);
    e >>= 1;
  }
  return r;
}
```

4.12 Combinatoria

```
// Factoriales e inversos modulo un primo. Precalculo O(n),
// C(n, k) en O(1). Requiere n < mod.
struct Combinatorics {
  long long mod;
  vector<long long> fact, inv_fact;
  Combinatorics(int n, long long mod_)
      : mod(mod_), fact(n + 1, 1), inv_fact(n + 1, 1) {
    for (int i = 1; i <= n; i++) fact[i] = fact[i - 1] * i % mod;
    inv_fact[n] = power(fact[n], mod - 2);
    for (int i = n; i >= 1; i--)
      inv_fact[i - 1] = inv_fact[i] * i % mod;
  }
  long long power(long long b, long long e) {
    long long r = 1;
    for (b %= mod; e > 0; e >>= 1) {
      if (e & 1) r = r * b % mod;
      b = b * b % mod;
    }
    return r;
  }
  long long C(int n, int k) {
    if (k < 0 || k > n) return 0;
    return fact[n] * inv_fact[k] % mod * inv_fact[n - k] % mod;
```

```
  }
};
```

5. Strings

5.1 Hashing

```
// Hashing polinomial modulo 2^61 - 1 con base aleatoria
// (resistente a anti-hash). Construccion O(n), consulta O(1).
// get(l, r): hash de s[l..r), comparable entre strings distintos.
struct Hashing {
  static const long long MOD = (1LL << 61) - 1;
  static long long mul(long long a, long long b) {
    return (long long)((__int128)a * b % MOD);
  }
  static long long base() {
    static mt19937_64 gen(
        chrono::steady_clock::now().time_since_epoch().count());
    static long long b = 256 + gen() % (MOD - 1000);
    return b;
  }
  vector<long long> h, pw;
  Hashing(const string& s) : h(s.size() + 1, 0),
                             pw(s.size() + 1, 1) {
    for (int i = 0; i < (int)s.size(); i++) {
      h[i + 1] = (mul(h[i], base()) + (unsigned char)s[i]) % MOD;
      pw[i + 1] = mul(pw[i], base());
    }
  }
  long long get(int l, int r) {  // [l, r)
    return (h[r] - mul(h[l], pw[r - l]) + MOD) % MOD;
  }
};
```

5.2 KMP

```
// KMP. O(n + m).
// prefix_function(s)[i] = largo del mayor prefijo propio de s[0..i] que
// tambien es sufijo de s[0..i].
vector<int> prefix_function(const string& s) {
  int n = s.size();
  vector<int> pi(n, 0);
  for (int i = 1; i < n; i++) {
    int j = pi[i - 1];
    while (j > 0 && s[i] != s[j]) j = pi[j - 1];
    if (s[i] == s[j]) j++;
    pi[i] = j;
  }
  return pi;
}
// Posiciones iniciales de todas las apariciones de pat en text.
vector<int> kmp_search(const string& text, const string& pat) {
  vector<int> pi = prefix_function(pat), res;
  int m = pat.size(), j = 0;
  if (m == 0) return res;
  for (int i = 0; i < (int)text.size(); i++) {
    while (j > 0 && text[i] != pat[j]) j = pi[j - 1];
    if (text[i] == pat[j]) j++;
    if (j == m) {
      res.push_back(i - m + 1);
      j = pi[j - 1];
    }
  }
  return res;
}
```

5.3 Función Z

```
// Funcion Z. O(n). z[i] = largo del mayor prefijo comun entre s
// y s[i..]. Por convencion z[0] = n.
// Buscar pat en text: calcular Z de pat + "#" + text y mirar
// las posiciones con z[i] == largo de pat.
vector<int> z_function(const string& s) {
  int n = s.size();
  vector<int> z(n, 0);
  if (n > 0) z[0] = n;
  for (int i = 1, l = 0, r = 0; i < n; i++) {
    if (i < r) z[i] = min(r - i, z[i - l]);
    while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
    if (i + z[i] > r) {
      l = i;
      r = i + z[i];
    }
  }
  return z;
}
```

5.4 Manacher

```
// Manacher: todos los palindromos en O(n).
// odd[i] = k: el mayor palindromo impar centrado en i es
//             s[i-k+1 .. i+k-1] (largo 2k - 1).
// even[i] = k:   el mayor palindromo par centrado entre i-1 e i
//             es s[i-k .. i+k-1] (largo 2k).
// Cantidad total de substrings palindromos = suma de ambos.
void manacher(const string& s, vector<int>& odd,
              vector<int>& even) {
  int n = s.size();
  odd.assign(n, 0);
  even.assign(n, 0);
  for (int i = 0, l = 0, r = -1; i < n; i++) {
    int k = i > r ? 1 : min(odd[l + r - i], r - i + 1);
    while (i - k >= 0 && i + k < n && s[i - k] == s[i + k]) k++;
    odd[i] = k;
    if (i + k - 1 > r) {
      l = i - k + 1;
      r = i + k - 1;
    }
  }
  for (int i = 0, l = 0, r = -1; i < n; i++) {
    int k = i > r ? 0 : min(even[l + r - i + 1], r - i + 1);
    while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k])
      k++;
    even[i] = k;
```

```
    if (i + k - 1 > r) {
      l = i - k;
      r = i + k - 1;
    }
  }
}
```

5.5 Suffix Automaton (DP en el DAG)

```
// Suffix Automaton (letras 'a'..'z'). Construccion O(n * 26).
// Es un DAG donde cada camino desde el estado 0 es un substring
// distinto, asi que muchas preguntas son una DP sobre el DAG.
// A lo sumo 2n estados. Cada estado v representa los substrings
// de largo (len[link[v]], len[v]] con el mismo conjunto de
// posiciones finales.
struct SuffixAutomaton {
  struct State {
    int len = 0, link = -1;
    long long cnt = 0;  // apariciones (tras compute_occurrences)
    array<int, 26> next;
    State() { next.fill(-1); }
  };
  vector<State> st;
  int last = 0;
  SuffixAutomaton(const string& s = "") : st(1) {
    for (char c : s) extend(c);
  }
  void extend(char ch) {
    int c = ch - 'a', cur = st.size();
    st.emplace_back();
    st[cur].len = st[last].len + 1;
    st[cur].cnt = 1;
    int p = last;
    while (p != -1 && st[p].next[c] == -1) {
      st[p].next[c] = cur;
      p = st[p].link;
    }
    if (p == -1) {
      st[cur].link = 0;
    } else {
      int q = st[p].next[c];
      if (st[p].len + 1 == st[q].len) {
        st[cur].link = q;
      } else {
        int clone = st.size();
        State cloned = st[q];
        cloned.len = st[p].len + 1;
        cloned.cnt = 0;
        st.push_back(cloned);
        while (p != -1 && st[p].next[c] == q) {
          st[p].next[c] = clone;
          p = st[p].link;
        }
        st[q].link = clone;
        st[cur].link = clone;
      }
    }
    last = cur;
  }
  // Estados por len creciente = orden topologico del DAG.
  vector<int> by_len() {
    vector<int> order(st.size());
    iota(order.begin(), order.end(), 0);
    sort(order.begin(), order.end(),
         [&](int a, int b) { return st[a].len < st[b].len; });
    return order;
  }
  // Deja en cnt[v] cuantas veces aparecen los substrings de v.
  void compute_occurrences() {
    vector<int> order = by_len();
    for (int i = (int)order.size() - 1; i > 0; i--)
      st[st[order[i]].link].cnt += st[order[i]].cnt;
  }
  long long occurrences(const string& t) {
    int v = 0;
    for (char ch : t) {
      v = st[v].next[ch - 'a'];
      if (v == -1) return 0;
    }
    return st[v].cnt;
  }
  // Cantidad de substrings distintos (no vacios).
  long long distinct() {
    long long r = 0;
    for (int v = 1; v < (int)st.size(); v++)
      r += st[v].len - st[st[v].link].len;
    return r;
  }
  // DP en el DAG: dp[v] = caminos que salen de v (incluye vacio).
  vector<long long> paths() {
    vector<int> order = by_len();
    vector<long long> dp(st.size(), 1);
    for (int i = (int)order.size() - 1; i >= 0; i--)
      for (int u : st[order[i]].next)
        if (u != -1) dp[order[i]] += dp[u];
    return dp;
  }
  // k-esimo substring distinto en orden lexicografico (k >= 1).
  // Retorna "" si hay menos de k.
  string kth(long long k) {
    vector<long long> dp = paths();
    if (k > dp[0] - 1) return "";
    string res;
    int v = 0;
    while (k > 0)
      for (int c = 0; c < 26; c++) {
        int u = st[v].next[c];
        if (u == -1) continue;
        if (k <= dp[u]) {
          res += (char)('a' + c);
```

```
        v = u;
        k--;
        break;
      }
      k -= dp[u];
    }
  return res;
 }
 // Largo del substring comun mas largo con t. O(|t|).
 int lcs(const string& t) {
  int v = 0, l = 0, best = 0;
  for (char ch : t) {
   int c = ch - 'a';
   while (v != 0 && st[v].next[c] == -1) {
    v = st[v].link;
    l = st[v].len;
   }
   if (st[v].next[c] != -1) {
    v = st[v].next[c];
    l++;
   }
   best = max(best, l);
  }
  return best;
 }
};
```

5.6 Suffix Array y LCP

```
// Suffix Array en O(n log n) (duplicacion + counting sort) y
// arreglo LCP de Kasai en O(n).
// sa[i] = inicio del i-esimo sufijo en orden lexicografico.
// lcp[i] = prefijo comun mas largo entre sa[i] y sa[i + 1].
// Substrings distintos = n (n + 1) / 2 - suma de lcp.
vector<int> suffix_array(const string& t) {
 string s = t;
 s.push_back('\0');  // centinela menor que todo
 int n = s.size(), classes = 1;
 vector<int> p(n), c(n), pn(n), cn(n), cnt(max(n, 256), 0);
 for (int i = 0; i < n; i++) cnt[(unsigned char)s[i]]++;
 for (int i = 1; i < 256; i++) cnt[i] += cnt[i - 1];
 for (int i = 0; i < n; i++) p[--cnt[(unsigned char)s[i]]] = i;
 c[p[0]] = 0;
 for (int i = 1; i < n; i++) {
  if (s[p[i]] != s[p[i - 1]]) classes++;
  c[p[i]] = classes - 1;
 }
 for (int h = 0; (1 << h) < n; h++) {
  int step = 1 << h;
  for (int i = 0; i < n; i++) pn[i] = (p[i] - step + n) % n;
  fill(cnt.begin(), cnt.begin() + classes, 0);
  for (int i = 0; i < n; i++) cnt[c[pn[i]]]++;
  for (int i = 1; i < classes; i++) cnt[i] += cnt[i - 1];
  for (int i = n - 1; i >= 0; i--) p[--cnt[c[pn[i]]]] = pn[i];
  cn[p[0]] = 0;
  classes = 1;
  for (int i = 1; i < n; i++) {
   pair<int, int> a = {c[p[i]], c[(p[i] + step) % n]};
   pair<int, int> b = {c[p[i - 1]], c[(p[i - 1] + step) % n]};
   if (a != b) classes++;
   cn[p[i]] = classes - 1;
  }
  c.swap(cn);
 }
 p.erase(p.begin());  // quitar el centinela
 return p;
}
vector<int> lcp_kasai(const string& s, const vector<int>& sa) {
 int n = s.size(), k = 0;
 vector<int> rnk(n), lcp(max(n - 1, 0));
 for (int i = 0; i < n; i++) rnk[sa[i]] = i;
 for (int i = 0; i < n; i++) {
  if (rnk[i] == n - 1) { k = 0; continue; }
  int j = sa[rnk[i] + 1];
  while (i + k < n && j + k < n && s[i + k] == s[j + k]) k++;
  lcp[rnk[i]] = k;
  if (k > 0) k--;
 }
 return lcp;
}
```

5.7 Aho-Corasick

```
// Aho-Corasick (letras 'a'..'z'): busca muchos patrones a la vez.
// Uso: insert() cada patron, build(), luego count_matches(text).
// O(suma de largos * 26 + |text|).
struct AhoCorasick {
 struct Node {
  array<int, 26> next;
  int link = 0;
  Node() { next.fill(-1); }
 };
 vector<Node> t;
 vector<int> order;        // nodos en orden BFS (sin la raiz)
 vector<int> pattern_node;  // nodo donde termina cada patron
 AhoCorasick() : t(1) {}
 int insert(const string& s) {  // retorna el id del patron
  int v = 0;
  for (char ch : s) {
   int c = ch - 'a';
   if (t[v].next[c] == -1) {
    int new_node = t.size();
    t.emplace_back();
    t[v].next[c] = new_node;
   }
   v = t[v].next[c];
  }
  pattern_node.push_back(v);
  return (int)pattern_node.size() - 1;
 }
 void build() {
  queue<int> q;
```

```
    for (int c = 0; c < 26; c++) {
      int u = t[0].next[c];
      if (u == -1) t[0].next[c] = 0;
      else q.push(u);
    }
    while (!q.empty()) {
      int v = q.front();
      q.pop();
      order.push_back(v);
      for (int c = 0; c < 26; c++) {
        int u = t[v].next[c];
        if (u == -1) {
          t[v].next[c] = t[t[v].link].next[c];
        } else {
          t[u].link = t[t[v].link].next[c];
          q.push(u);
        }
      }
    }
  }
  // res[i] = cantidad de apariciones del patron i en el texto.
  vector<long long> count_matches(const string& text) {
    vector<long long> visits(t.size(), 0);
    int v = 0;
    for (char ch : text) {
      v = t[v].next[ch - 'a'];
      visits[v]++;
    }
    for (int i = (int)order.size() - 1; i >= 0; i--)
      visits[t[order[i]].link] += visits[order[i]];
    vector<long long> res;
    for (int node : pattern_node) res.push_back(visits[node]);
    return res;
  }
};
```

6. Geometría

6.1 Punto y primitivas

```
// Punto entero (exacto). Con coordenadas hasta 1e9 los productos
// caben en long long. PtD: version real para circulos.
struct Pt {
  long long x, y;
  Pt operator+(Pt o) const { return {x + o.x, y + o.y}; }
  Pt operator-(Pt o) const { return {x - o.x, y - o.y}; }
  long long cross(Pt o) const { return x * o.y - y * o.x; }
  long long dot(Pt o) const { return x * o.x + y * o.y; }
  long long norm2() const { return x * x + y * y; }
  bool operator<(Pt o) const {
    return x != o.x ? x < o.x : y < o.y;
  }
  bool operator==(Pt o) const { return x == o.x && y == o.y; }
};
struct PtD { double x, y; };
```

```
// > 0: c a la izquierda de a->b.  < 0: derecha.  0: colineales.
long long orient(Pt a, Pt b, Pt c) { return (b - a).cross(c - a); }
int sign(long long v) { return (v > 0) - (v < 0); }
// p esta sobre el segmento cerrado ab?
bool on_segment(Pt a, Pt b, Pt p) {
  return orient(a, b, p) == 0 && (a - p).dot(b - p) <= 0;
}
// Doble del area con signo (positiva si es antihorario).
long long area2(const vector<Pt>& poly) {
  long long s = 0;
  int n = poly.size();
  for (int i = 0; i < n; i++) s += poly[i].cross(poly[(i + 1) % n]);
  return s;
}
```

6.2 Intersección de segmentos

```
// Interseccion de segmentos. Requiere Pt, orient, sign,
// on_segment. Exacto con enteros, O(1).
// Los segmentos cerrados ab y cd comparten algun punto?
bool segments_intersect(Pt a, Pt b, Pt c, Pt d) {
  int o1 = sign(orient(a, b, c)), o2 = sign(orient(a, b, d));
  int o3 = sign(orient(c, d, a)), o4 = sign(orient(c, d, b));
  if (o1 * o2 < 0 && o3 * o4 < 0) return true;  // cruce propio
  return on_segment(a, b, c) || on_segment(a, b, d) ||
         on_segment(c, d, a) || on_segment(c, d, b);
}
// Punto de interseccion de las RECTAS ab y cd.
// Retorna false si son paralelas (o coincidentes).
bool line_intersection(Pt a, Pt b, Pt c, Pt d, PtD& res) {
  long long den = (b - a).cross(d - c);
  if (den == 0) return false;
  double t = (double)(c - a).cross(d - c) / (double)den;
  res = {a.x + t * (b.x - a.x), a.y + t * (b.y - a.y)};
  return true;
}
```

6.3 Convex Hull

```
// Convex Hull (Andrew, cadena monotona). O(n log n). Requiere Pt.
// Retorna el poligono en sentido antihorario, sin puntos
// colineales ni repetidos. Para conservar los colineales del
// borde cambiar <= 0 por < 0.
vector<Pt> convex_hull(vector<Pt> p) {
  sort(p.begin(), p.end());
  p.erase(unique(p.begin(), p.end()), p.end());
  if (p.size() < 3) return p;
  vector<Pt> h;
  for (int pass = 0; pass < 2; pass++) {
    int base = h.size();
    for (Pt q : p) {
      while ((int)h.size() >= base + 2 &&
             orient(h[h.size() - 2], h.back(), q) <= 0)
        h.pop_back();
      h.push_back(q);
    }
    h.pop_back();
```

```
    reverse(p.begin(), p.end());
  }
  return h;
}
```

6.4 Puntos antipodales (rotating calipers)

```
// Pares antipodales con rotating calipers. O(n). Requiere Pt.
// h = poligono estrictamente convexo en sentido antihorario
// (la salida de convex_hull). El diametro (par de puntos mas
// lejano) siempre es uno de los pares devueltos.
vector<pair<int, int>> antipodal_pairs(const vector<Pt>& h) {
  int n = h.size();
  vector<pair<int, int>> pairs;
  if (n < 2) return pairs;
  if (n == 2) return {{0, 1}};
  for (int i = 0, j = 1; i < n; i++) {
    Pt side = h[(i + 1) % n] - h[i];
    // avanzar j mientras se aleje de la recta del lado i
    while (side.cross(h[(j + 1) % n] - h[j]) > 0) j = (j + 1) % n;
    pairs.push_back({i, j});
    pairs.push_back({(i + 1) % n, j});
  }
  return pairs;
}
// Cuadrado de la mayor distancia entre dos puntos del poligono.
long long diameter2(const vector<Pt>& h) {
  long long best = 0;
  for (auto [i, j] : antipodal_pairs(h))
    best = max(best, (h[i] - h[j]).norm2());
  return best;
}
```

6.5 Círculo mínimo

```
// Circulo minimo que encierra un conjunto de puntos (Welzl
// incremental aleatorizado). O(n) esperado. Requiere PtD.
struct Circle { double x, y, r; };

bool contains(const Circle& c, PtD p) {
  return hypot(p.x - c.x, p.y - c.y) <= c.r + 1e-9;
}
Circle from_two(PtD a, PtD b) {
  return {(a.x + b.x) / 2, (a.y + b.y) / 2,
          hypot(a.x - b.x, a.y - b.y) / 2};
}
Circle from_three(PtD a, PtD b, PtD c) {  // circunscrito
  double bx = b.x - a.x, by = b.y - a.y;
  double cx = c.x - a.x, cy = c.y - a.y;
  double d = 2 * (bx * cy - by * cx);
  double b2 = bx * bx + by * by, c2 = cx * cx + cy * cy;
  double ux = (cy * b2 - by * c2) / d;
  double uy = (bx * c2 - cx * b2) / d;
  return {a.x + ux, a.y + uy, hypot(ux, uy)};
}
Circle min_enclosing_circle(vector<PtD> p) {
  shuffle(p.begin(), p.end(), mt19937(712));
  int n = p.size();
  Circle c = {p[0].x, p[0].y, 0};
  for (int i = 1; i < n; i++) {
    if (contains(c, p[i])) continue;
    c = {p[i].x, p[i].y, 0};
    for (int j = 0; j < i; j++) {
      if (contains(c, p[j])) continue;
      c = from_two(p[i], p[j]);
      for (int k = 0; k < j; k++)
        if (!contains(c, p[k])) c = from_three(p[i], p[j], p[k]);
    }
  }
  return c;
}
```

6.6 Punto en polígono

```
// Punto en poligono. Requiere Pt, orient, on_segment.
// Ambas retornan 0 = fuera, 1 = en el borde, 2 = dentro.

// Poligono simple cualquiera (convexo o no). O(n).
int in_polygon(const vector<Pt>& poly, Pt p) {
  int n = poly.size();
  bool inside = false;
  for (int i = 0; i < n; i++) {
    Pt a = poly[i], b = poly[(i + 1) % n];
    if (on_segment(a, b, p)) return 1;
    if (a.y > b.y) swap(a, b);
    // el rayo horizontal hacia la derecha cruza el lado ab?
    if (a.y <= p.y && p.y < b.y && orient(a, b, p) > 0)
      inside = !inside;
  }
  return inside ? 2 : 0;
}

// Poligono estrictamente convexo en sentido antihorario con
// n >= 3 (la salida de convex_hull). O(log n).
int in_convex(const vector<Pt>& h, Pt p) {
  int n = h.size();
  long long o_first = orient(h[0], h[1], p);
  long long o_last = orient(h[0], h[n - 1], p);
  if (o_first < 0 || o_last > 0) return 0;
  int lo = 1, hi = n - 1;
  while (hi - lo > 1) {
    int mid = (lo + hi) / 2;
    if (orient(h[0], h[mid], p) >= 0) lo = mid;
    else hi = mid;
  }
  long long o = orient(h[lo], h[hi], p);
  if (o < 0) return 0;
  if (o == 0) return 1;
  if (lo == 1 && o_first == 0) return 1;
  if (hi == n - 1 && o_last == 0) return 1;
  return 2;
}
```

6.7 Par de puntos más cercano

```cpp
// Par de puntos mas cercano, divide y venceras. O(n log n).
// Requiere Pt. Retorna el CUADRADO de la distancia minima
// (n >= 2). Coordenadas hasta 1e9.
long long closest_rec(vector<Pt>& p, vector<Pt>& tmp, int l,
                      int r) {  // rango [l, r)
  auto by_y = [](Pt a, Pt b) { return a.y < b.y; };
  if (r - l <= 3) {
    long long d = LLONG_MAX;
    for (int i = l; i < r; i++)
      for (int j = i + 1; j < r; j++)
        d = min(d, (p[i] - p[j]).norm2());
    sort(p.begin() + l, p.begin() + r, by_y);
    return d;
  }
  int mid = (l + r) / 2;
  long long xm = p[mid].x;
  long long d = min(closest_rec(p, tmp, l, mid),
                    closest_rec(p, tmp, mid, r));
  merge(p.begin() + l, p.begin() + mid, p.begin() + mid,
        p.begin() + r, tmp.begin(), by_y);
  copy(tmp.begin(), tmp.begin() + (r - l), p.begin() + l);
  int k = 0;  // tmp[0..k) = franja central, ordenada por y
  for (int i = l; i < r; i++) {
    long long dx = p[i].x - xm;
    if (dx * dx >= d) continue;
    for (int j = k - 1; j >= 0; j--) {
      long long dy = p[i].y - tmp[j].y;
      if (dy * dy >= d) break;
      d = min(d, (p[i] - tmp[j]).norm2());
    }
    tmp[k++] = p[i];
  }
  return d;
}
long long closest_pair(vector<Pt> p) {
  sort(p.begin(), p.end());
  vector<Pt> tmp(p.size());
  return closest_rec(p, tmp, 0, p.size());
}
```

7. Fórmulas y recordatorios

Límites. `int` $\approx 2.1 \cdot 10^9$, `long long` $\approx 9.2 \cdot 10^{18}$, `__int128` $\approx 1.7 \cdot 10^{38}$, `double` 15–16 dígitos. Unas 10^8 operaciones simples por segundo. Guía por tamaño: $n \le 10$: $O(n!)$; $n \le 20$: $O(2^n n)$; $n \le 500$: $O(n^3)$; $n \le 5000$: $O(n^2)$; $n \le 10^6$: $O(n \log n)$; más: $O(n)$.

Primos útiles. 10^9+7, 10^9+9, $998244353 = 119 \cdot 2^{23}+1$, $2^{61}-1$. Máximo de divisores: 240 para $n \le 10^6$, 1344 para $n \le 10^9$, 103680 para $n \le 10^{18}$.

Sumas. $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$, $\sum i^2 = \frac{n(n+1)(2n+1)}{6}$, $\sum i^3 = \left(\frac{n(n+1)}{2}\right)^2$, $\sum_{i=0}^{n} r^i = \frac{r^{n+1}-1}{r-1}$, $\sum_{i=1}^{n} \frac{1}{i} \approx \ln n + 0.577$ (por eso $\sum_i n/i = O(n \log n)$).

Combinatoria.

- $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$; $\sum_k \binom{n}{k} = 2^n$.
- Soluciones de $x_1 + \cdots + x_k = n$ con $x_i \ge 0$: $\binom{n+k-1}{k-1}$.
- Catalan: $C_n = \frac{1}{n+1}\binom{2n}{n}$: 1, 1, 2, 5, 14, 42, 132, 429.
- Desarreglos: $D_n = (n-1)(D_{n-1} + D_{n-2})$, $D_0 = 1$, $D_1 = 0$.
- Lucas (p primo): $\binom{n}{k} \equiv \prod \binom{n_i}{k_i} \pmod p$ con n_i, k_i los dígitos en base p.
- Burnside: $\#\text{órbitas} = \frac{1}{|G|}\sum_{g \in G} |\text{Fix}(g)|$.
- Inclusión-exclusión: $|\bigcup A_i| = \sum_{S \neq \emptyset} (-1)^{|S|+1} |\bigcap_{i \in S} A_i|$.

Teoría de números.

- Fermat: $a^{p-1} \equiv 1 \pmod p$. Euler: $a^{\varphi(m)} \equiv 1 \pmod m$ si $\gcd(a, m) = 1$.
- $\varphi(n) = n \prod_{p|n} (1 - \frac{1}{p})$; $\sum_{d|n} \varphi(d) = n$.
- Wilson: $(p-1)! \equiv -1 \pmod p$.
- $ax + by = c$ tiene solución entera si y solo si $\gcd(a, b) \mid c$.
- $\text{lcm}(a, b) = a / \gcd(a, b) \cdot b$.

Grafos.

- Flujo máximo = corte mínimo.
- Kőnig (bipartito): cobertura mínima de vértices = matching máximo; independiente máximo = $V-$ matching.
- Cubrimiento mínimo por caminos disjuntos en un DAG = $V-$ matching (grafo bipartito salida/entrada).
- Dilworth: anticadena máxima = mínimo número de cadenas que cubren el orden parcial.
- Hall: hay matching perfecto del lado A si y solo si $|N(S)| \ge |S|$ para todo $S \subseteq A$.
- Euleriano, no dirigido: conexo y todos los grados pares (circuito) o exactamente dos impares (camino). Dirigido: grado de entrada = grado de salida.
- Euler (planar conexo): $V - E + F = 2$.
- Cayley: n^{n-2} árboles etiquetados. Kirchhoff: #árboles generadores = cualquier cofactor del laplaciano.
- 2-SAT: $(a \vee b) \equiv (\neg a \to b) \wedge (\neg b \to a)$.

Strings.

- Período mínimo de s: $n - \pi[n-1]$ (es período exacto si divide a n).
- Substrings distintos: $\frac{n(n+1)}{2} - \sum \text{lcp}$, o $\sum_v \text{len}(v) - \text{len}(\text{link}(v))$ en el suffix automaton.

Geometría.

- Área de polígono: $\frac{1}{2}\left|\sum x_i y_{i+1} - x_{i+1} y_i\right|$.
- Pick: $A = I + \frac{B}{2} - 1$; puntos enteros en un segmento: $\gcd(|dx|, |dy|) + 1$.
- Distancia de p a la recta ab: $\frac{|(b-a) \times (p-a)|}{|b-a|}$.
- Herón: $A = \sqrt{s(s-a)(s-b)(s-c)}$, $s = \frac{a+b+c}{2}$. Circunradio $R = \frac{abc}{4A}$, inradio $r = \frac{A}{s}$.
- Cosenos: $c^2 = a^2 + b^2 - 2ab\cos C$. Senos: $\frac{a}{\sin A} = 2R$.
- Rotar θ: $(x\cos\theta - y\sin\theta,\ x\sin\theta + y\cos\theta)$.

Antes de enviar.

- ¿Desborda `int`? Usar `long long` en productos y sumas.
- Casos borde: $n = 1$, vacío, todos iguales, grafo no conexo.
- Reiniciar variables globales entre casos de prueba.
- Módulo de negativos: `((x % m) + m) % m`.
- Recursión profunda ($n \ge 10^5$ en cadena): cuidado con la pila.
- Usar `'\n'` en vez de `endl`.
- Comparar reales con tolerancia (`1e-9`); preferir enteros en geometría.

Referencias para ampliar. KACTL (KTH), Biblioteca de la UFMG, El Vasito (UNC), notebook de Stanford, cp-algorithms.com.